

VIETNAMESE STUDENT HACKAITHON 2026
BẢNG C - INNOVATOR

★ ☆ ★



FINAL SUBMISSION DOCUMENTATION

Private Test Docker Package and Method Report

FASTMCQ AGENT

Offline Vietnamese Multiple-Choice Reasoning System

Docker-first Inference with Local Qwen3-4B

Author: Võ Quốc Linh
Submission track: Bảng C - Innovator, Vòng 2
Docker image: vquclinh/fastmcq-agent:latest
Core model: Qwen/Qwen3-4B-Instruct-2507
Runtime mode: Offline local GPU inference
Target input: /code/private_test.json
Target outputs: /code/submission.csv, /code/submission_time.csv
Ho Chi Minh City, Ngày 28 tháng 6 năm 2026

Mục lục

1	Tóm tắt điều hành	2
1.1	Định hướng final	2
1.2	Điểm nổi bật	2
2	Bối cảnh cuộc thi và yêu cầu nộp bài	2
3	Kiến trúc tổng thể	3
3.1	Sơ đồ final runtime path	3
3.2	Luồng xử lý chi tiết	3
4	Hệ sinh thái module trong repository	4
4.1	Kiến trúc dynamic full-system đã phát triển	4
5	Final model strategy	4
5.1	Model được chọn	4
5.2	Vì sao chọn Qwen3-4B	5
5.3	Deterministic inference	5
6	Data processing	5
6.1	Input format	5
6.2	Output format	5
7	Docker build và runtime	6
7.1	Dockerfile design	6
7.2	Build-time resource initialization	6
7.3	Runtime commands	6
8	Tính đúng đắn và kiểm thử	6
8.1	Static validation	6
8.2	Runtime validation đề xuất	7
9	Các điểm mạnh kỹ thuật	7
9.1	Tuân thủ chặt contract	7
9.2	Không phụ thuộc external service	7
9.3	Kiến trúc module hóa	7
9.4	Defensive output handling	7
9.5	Per-sample timing thật	7
10	Rủi ro và biện pháp giảm thiểu	7
11	Kết luận	7
	Tài liệu tham khảo	8

1 Tóm tắt điều hành

FASTMCQ Agent là hệ thống trả lời câu hỏi trắc nghiệm tiếng Việt được thiết kế cho vòng private test của Vietnamese Student HackAlthon 2026 - Bảng C Innovator. Hệ thống được đóng gói dưới dạng Docker image và có mục tiêu chính: đọc bộ kiểm thử tại `/code/private_test.json`, suy luận từng câu hỏi, và xuất hai file đúng chuẩn BTC ngay trong `/code: submission.csv` và `submission_time.csv`.

1.1 Định hướng final

Phiên bản final chuyển sang hướng **fully offline local-model inference**. Điều này phù hợp với bối cảnh chấm điểm có thể bị giới hạn internet và tránh phụ thuộc external API. Model được chọn là **Qwen/Qwen3-4B-Instruct-2507**, một model instruct dense 4.0B parameters theo model card, thuộc họ Qwen3 và có thể chạy qua Hugging Face Transformers [R1, R2]. Hệ thống dùng một model open-weight duy nhất trong giới hạn yêu cầu mô hình nhỏ hơn hoặc bằng 5B.

1.2 Điểm nổi bật

- **Docker-first**: Dockerfile tự cài dependency, tải model ở build-time, và chạy `inference.sh` mặc định.
- **BTC I/O compliant**: input chính xác `/code/private_test.json`; output chính xác `/code/submission.csv` và `/code/submission_time.csv`.
- **Offline runtime**: model nằm trong image tại `/models/qwen3-4b-instruct-2507`; runtime đặt `TRANSFORMERS_OFFLINE=1` và `HF_HUB_OFFLINE=1`.
- **Per-sample timing**: hệ thống đo thời gian cho từng câu hỏi trong vòng lặp inference, không ghi thời gian trung bình của toàn bộ test.
- **Robust output**: parser ép output về nhãn hợp lệ theo số lượng lựa chọn; nếu model trả về không hợp lệ, fallback deterministic đảm bảo không mất dòng.

2 Bối cảnh cuộc thi và yêu cầu nộp bài

BTC yêu cầu mỗi đội cung cấp GitHub repository và Docker Hub image. Container phải chạy end-to-end, đọc dữ liệu được mount vào và sinh kết quả đúng định dạng. Theo xác nhận mới nhất từ BTC, các điểm quan trọng được hiểu như sau:

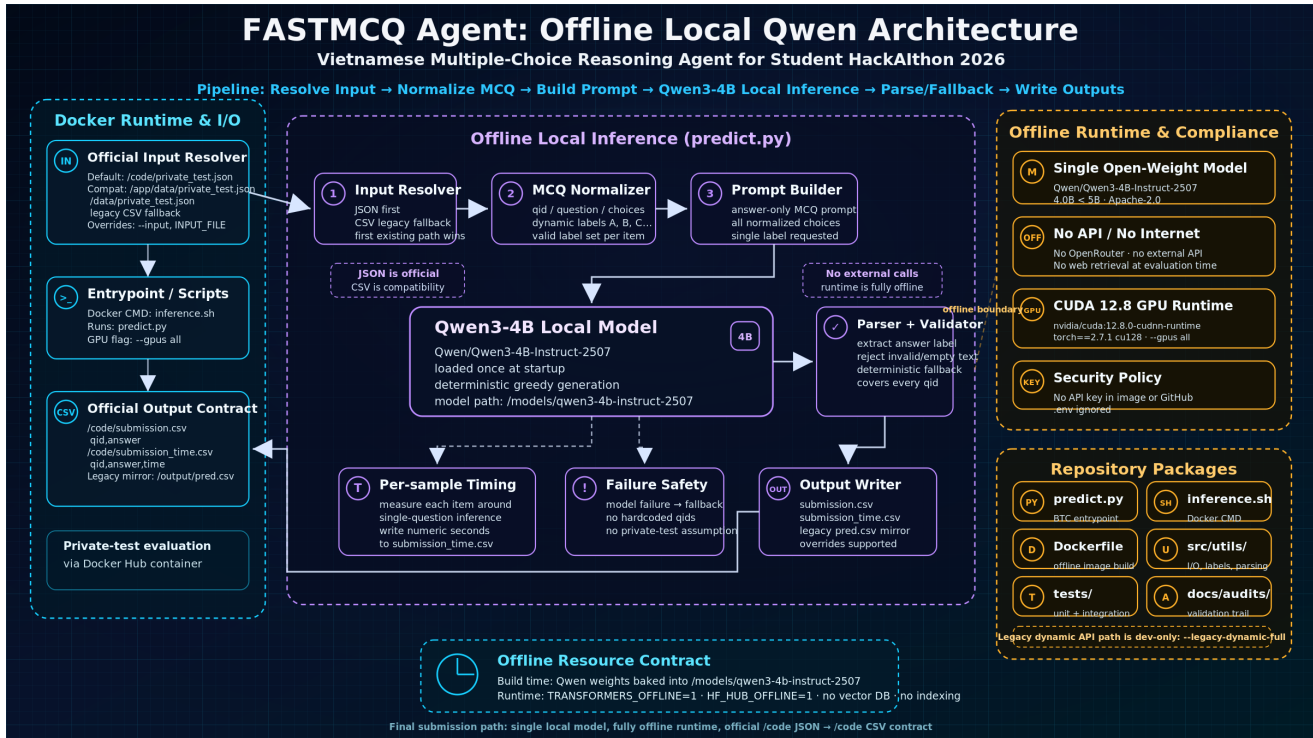
Yêu cầu	Cách FASTMCQ đáp ứng
Input chính thức	<code>/code/private_test.json</code> ; tên file chuẩn là <code>private_test.json</code> .
Output chính thức	Ghi ngay trong <code>/code: submission.csv</code> và <code>submission_time.csv</code> .
Entry point	Root <code>predict.py</code> là entry point; <code>inference.sh</code> gọi <code>pythonpredict.py</code> \protect\T5\textdollar@".
Docker default	<code>CMD["bash", "inference.sh"]</code> ; chạy container không cần command phụ.
Timing	Mỗi sample được đo thời gian riêng trong loop inference và dump vào <code>submission_time.csv</code> .
CUDA	Theo xác nhận bổ sung, target dùng CUDA 12.8+ cho RTX 5060 Ti; Dockerfile dùng <code>nvidia/cuda:12.8.0-cudnn-runtime-ubuntu22.04</code> .

Bảng 1: Mapping yêu cầu BTC với hiện thực trong FASTMCQ Agent.

3 Kiến trúc tổng thể

FASTMCQ Agent được thiết kế theo hai lớp nhìn: (1) **final runtime path** dùng offline local Qwen để đáp ứng chuẩn BTC, và (2) **module ecosystem** trong repository, bao gồm các module base prediction, dynamic layers, selector, solvers, evidence và API runtime đã được phát triển trong quá trình tối ưu. Final path không gọi external API, nhưng thừa hưởng triết lý thiết kế quan trọng: input normalization rõ ràng, all-qid output, parser an toàn, và khả năng kiểm soát output contract.

3.1 Sơ đồ final runtime path



Hình 1: Final runtime architecture of FASTMCQ Agent: the system reads the official BTC private-test input, normalizes samples, performs offline local Qwen3-4B inference, validates answer labels, measures per-sample runtime, and writes the required submission files.

3.2 Luồng xử lý chi tiết

- 1. Resolve input:** ưu tiên --input hoặc INPUT_FILE, sau đó /code/private_test.json. Các path tương thích như /app/data/private_test.json, /data/private_test.csv được giữ để test/reproduce linh hoạt.
- 2. Normalize sample:** mỗi câu được chuẩn hóa thành qid, question, và danh sách choices. Hệ thống không hardcode qid hay answer.
- 3. Load model once:** predictor tải model một lần từ LOCAL_MODEL_PATH; inference từng câu dùng cùng object model để tránh overhead lặp.
- 4. Predict per item:** với từng sample, hệ thống bắt đầu timer, tạo prompt answer-only, gọi model deterministic generation, rồi parse nhãn.
- 5. Validate label:** nhãn phải nằm trong không gian lựa chọn của câu hỏi. Nếu output không hợp lệ, fallback chọn nhãn đầu tiên để đảm bảo submission hoàn chỉnh.
- 6. Write outputs:** submission.csv có header qid, answer; submission_time.csv có header qid, answer, time và thời gian từng sample.

4 Hệ sinh thái module trong repository

Dù final runtime mặc định chạy local model offline, repository đã được tổ chức như một hệ thống nhiều module. Việc tách module giúp hệ thống dễ kiểm thử, dễ thay backend suy luận, và duy trì các ý tưởng đã phát triển từ những phase trước.

Module	Package / File	Vai trò kỹ thuật
Final entry point	<code>predict.py</code>	Entry point theo BTC; điều phối input, model, per-sample timing, output writing.
Shell runner	<code>inference.sh</code>	Script mặc định trong Docker; forward mọi flag về <code>predict.py</code> .
Local model	<code>src/local_model/</code>	Backend Qwen3-4B qua Transformers; lazy import torch/transformers; deterministic answer generation.
System orchestration	<code>src/system/</code>	Module orchestration cho dynamic full-system prototype, production profile, pipeline flow.
Base prediction	<code>src/base/</code>	Các base solvers/predictors để đảm bảo all-qid coverage trong các phiên bản dynamic.
Dynamic layers	<code>src/layers/</code>	V12B/V13 selective reasoning, permutation debiasing, content-first và least-to-most passes trong prototype API path.
API runtime	<code>src/api/</code>	Client và model policy cho OpenRouter/dev legacy path; không dùng trong final default path.
Selector	<code>src/selector/</code>	Candidate ranking, conservative override, consistency checks cho dynamic path.
Solvers	<code>src/solvers/</code>	Symbolic/programmatic/formula-based reasoning helpers.
Evidence	<code>src/evidence/</code>	Evidence packing, reranking, sufficiency checks, option grounding.
Utilities	<code>src/utis/</code>	I/O, label handling, parsing, logging, postprocessing.
Docker build	<code>Dockerfile</code>	CUDA 12.8 base, torch cu128, dependencies, build-time model download, offline runtime env.

Bảng 2: Các module chính trong repository và vai trò của từng phần.

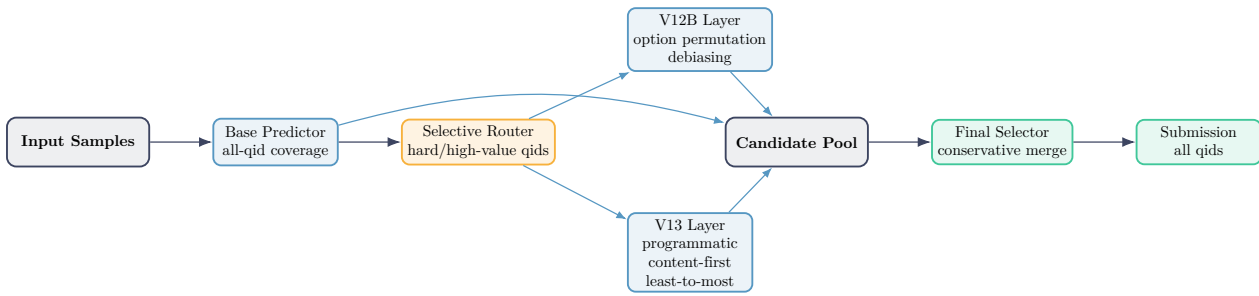
4.1 Kiến trúc dynamic full-system đã phát triển

Trong quá trình phát triển, FASTMCQ không chỉ là một single-shot solver. Hệ thống từng có dynamic full pipeline gồm Base Predictor, V12B option-permutation layer, V13 multi-layer reasoning, candidate pool và conservative selector. Kiến trúc này vẫn là một phần quan trọng của repository và tài liệu phương pháp, nhưng final BTC path không gọi external API để đảm bảo offline runtime.

5 Final model strategy

5.1 Model được chọn

Final image dùng **Qwen/Qwen3-4B-Instruct-2507**. Theo model card trên Hugging Face, model có 4.0B parameters, 3.6B non-embedding parameters, 36 layers, GQA attention heads, và context length lớn [R1]. Qwen3 technical report mô tả Qwen3 là họ LLM gồm dense và MoE models, hướng tới hiệu



Hình 2: Dynamic full-system architecture trong quá trình phát triển: selective reasoning và conservative candidate merge.

năng, hiệu quả và năng lực đa ngôn ngữ [R2]. Những đặc điểm này phù hợp với bài toán MCQ tiếng Việt trong điều kiện model-size cap.

5.2 Vì sao chọn Qwen3-4B

- **Tuân thủ giới hạn model:** 4.0B nằm dưới ngưỡng 5B.
- **Instruction-following:** dạng instruct phù hợp prompt MCQ answer-only.
- **Tính độc lập:** model được đóng gói trong image, không cần API runtime.
- **Cân bằng accuracy/time:** lớn hơn các model 0.5B/1.5B/3B, nhưng vẫn thực tế hơn so với model vượt cap hoặc cần server riêng.

5.3 Deterministic inference

Hệ thống cấu hình generation theo hướng ổn định: `do_sample=False`, `num_beams=1`, và `max_new_tokens` nhỏ. Prompt yêu cầu model trả về nhãn đáp án duy nhất. Sau đó parser chỉ nhận nhãn nằm trong label space hợp lệ của câu hỏi.

6 Data processing

6.1 Input format

Input chính thức là JSON tại `/code/private_test.json`. Mỗi item cần có `qid`, `question`, và các lựa chọn. Pipeline được thiết kế để chịu được khác biệt nhẹ về schema, ví dụ lựa chọn có thể ở dạng list choices hoặc các field A, B, C, D. Mục tiêu là normalize về một representation duy nhất trước khi tạo prompt.

6.2 Output format

Hệ thống ghi hai file:

```

/code/submission.csv
qid,answer
test_0001,A
test_0002,B
  
```

```

/code/submission_time.csv
qid,answer,time
test_0001,A,1.2345
test_0002,B,2.9087
  
```

Trường `time` là thời gian suy luận từng sample, đo quanh lời gọi `predict_one(item)` và `fallback/parse` cho sample đó. Đây không phải là total wall-clock chia trung bình cho số câu.

7 Docker build và runtime

7.1 Dockerfile design

Dockerfile final được thiết kế theo chuẩn BTC và theo xác nhận CUDA 12.8+:

```
FROM nvidia/cuda:12.8.0-cudnn-runtime-ubuntu22.04
WORKDIR /code
RUN apt-get update && apt-get install -y --no-install-recommends \
    python3 python3-pip python3-dev git ca-certificates
RUN python -m pip install --index-url https://download.pytorch.org/whl/cu128 torch==2.7.1
COPY requirements.txt .
RUN python -m pip install -r requirements.txt
COPY . /code
RUN python scripts/download_local_model.py --model Qwen/Qwen3-4B-Instruct-2507 \
    --out /models/qwen3-4b-instruct-2507
ENV LOCAL_MODEL_PATH=/models/qwen3-4b-instruct-2507 \
    TRANSFORMERS_OFFLINE=1 \
    HF_HUB_OFFLINE=1
CMD ["bash", "inference.sh"]
```

7.2 Build-time resource initialization

Không có vector database hoặc index ngoài. Tài nguyên duy nhất cần init là model weights. Script `scripts/download_local_model.py` dùng Hugging Face Hub để tải model trong quá trình build. Vì vậy image push lên Docker Hub đã chứa weights và tokenizer cần thiết.

7.3 Runtime commands

Lệnh chính thức để reproduce:

```
docker pull vquclinh/fastmcq-agent:latest

docker run --rm --gpus all \
    -v /path/to/private_test.json:/code/private_test.json:ro \
    vquclinh/fastmcq-agent:latest
```

Trong local smoke test, có thể override output để dễ lấy file ra host:

```
docker run --rm --gpus all --network none \
    -v "$PWD/btc_data/private_test.json:/code/private_test.json:ro" \
    -v "$PWD/btc_output:/code/btc_output" \
    -e SUBMISSION_FILE=/code/btc_output/submission.csv \
    -e SUBMISSION_TIME_FILE=/code/btc_output/submission_time.csv \
    vquclinh/fastmcq-agent:latest
```

8 Tính đúng đắn và kiểm thử

8.1 Static validation

Repository đã có các kiểm thử static/integration cho:

- tồn tại root Dockerfile, `predict.py`, `inference.sh`, `README.md`, `requirements.txt`;
- input priority có `/code/private_test.json`;
- output headers đúng `qid,answer` và `qid,answer,time`;
- timing từng sample được đo quanh từng lời gọi model;
- Dockerfile dùng CUDA 12.8 base, torch cu128, model download, offline env;
- secret/model weights không bị track vào GitHub.

8.2 Runtime validation đề xuất

Trước khi nộp, image nên được kiểm thử theo ba mức:

1. **Build test:** `dockerbuild-tvquclinh/fastmcq-agent:latest`.
2. **GPU smoke:** chạy input nhỏ với `--gpusall` để xác minh container thấy GPU.
3. **Offline smoke:** chạy với `--networknone` để đảm bảo model đã nằm trong image và runtime không gọi internet.

9 Các điểm mạnh kỹ thuật

9.1 Tuân thủ chặt contract

Hệ thống ưu tiên đúng contract trước: path input chính thức, output chính thức, default command, per-sample time, và image offline. Điều này giảm rủi ro hỏng bài nộp vì sai quy cách.

9.2 Không phụ thuộc external service

External API có thể mạnh nhưng không phù hợp môi trường chấm offline. Việc chuyển sang local model giúp submission tự chứa toàn bộ tài nguyên cần thiết. Điều này phù hợp nguyên tắc Docker image reproducible.

9.3 Kiến trúc module hóa

Repository không nhồi logic vào một file duy nhất. Các phần entry point, model backend, data I/O, solvers, selector, dynamic layers, API runtime và evidence đều được tách rõ. Nhờ vậy hệ thống có thể audit, test và thay backend mà không phá contract.

9.4 Defensive output handling

MCQ evaluation rất nhạy với format. Vì vậy parser và fallback giữ vai trò quan trọng: dù model trả lời dài, sai format hoặc không trả nhãn hợp lệ, hệ thống vẫn sinh đúng số dòng và đúng header.

9.5 Per-sample timing thật

Thời gian được đo từng item, đúng yêu cầu dump thời gian cho từng điểm dữ liệu. Điều này tốt hơn việc ghi một giá trị trung bình cho mọi câu.

10 Rủi ro và biện pháp giảm thiểu

11 Kết luận

FASTMCQ Agent final submission là một hệ thống MCQ reasoning được đóng gói theo hướng tự chứa, offline và Docker-first. Dù quá trình phát triển có nhiều lớp dynamic reasoning như Base Predictor, V12B, V13 và selector, phiên bản nộp final ưu tiên contract của BTC: một model open-weight local, build-time resource initialization, runtime không internet, output đúng định dạng và timing từng sample. Cách thiết kế này cân bằng giữa năng lực suy luận của Qwen3-4B, tính reproducible của Docker, và yêu cầu chấm tự động của cuộc thi.

Rủi ro	Biện pháp giảm thiểu
Image lớn	Model và CUDA runtime bắt buộc chiếm nhiều dung lượng; dùng Docker Hub image final và có thể backup Drive nếu form yêu cầu.
Local GPU yếu/OOM	Target chính thức là môi trường BTC; local RTX 4060 8GB có thể không phản ánh đúng BTC. Smoke test cần phân biệt lỗi image và lỗi VRAM local.
CUDA mismatch	Dùng base CUDA 12.8 và torch cu128 theo xác nhận BTC; không downgrade theo máy local.
Hugging Face download fail lúc build	Build cần network và đủ disk; có thể dùng HF token để tăng rate limit khi cần.
Output không hợp lệ	Parser/fallback đảm bảo mọi qid đều có answer label hợp lệ.

Bảng 3: Rủi ro thực tế và cách xử lý trong thiết kế final.

Tài liệu tham khảo

- [R1] Qwen Team. *Qwen/Qwen3-4B-Instruct-2507 Model Card*. Hugging Face. <https://huggingface.co/Qwen/Qwen3-4B-Instruct-2507>. Model card ghi 4.0B parameters, 3.6B non-embedding parameters và cấu hình model.
- [R2] Yang et al. *Qwen3 Technical Report*. arXiv:2505.09388, 2025. <https://arxiv.org/abs/2505.09388>.
- [R3] PyTorch. *Previous PyTorch Versions / CUDA wheel installation*. <https://pytorch.org/get-started/previous-versions/>.
- [R4] Hugging Face. *Transformers documentation*. <https://huggingface.co/docs/transformers>.
- [R5] Docker. *GPU support in Docker Desktop for Windows*. <https://docs.docker.com/desktop/features/gpu/>.
- [R6] Docker. *Docker Desktop WSL 2 backend on Windows*. <https://docs.docker.com/desktop/features/wsl/>.
- [R7] NVIDIA. *CUDA container images and compatibility guidance*. <https://hub.docker.com/r/nvidia/cuda>.